

# AI 開発ワークショップ

AIを活用した現代的なソフトウェア開発



## 自己紹介



Satoru Takahashi

X: @cmd8051020498/Satoru

GDGoC Organizer

### 所属

- 公立はこだて未来大学
- Dotto
- GDGoC Hakodate

### 好きな技術

- Flutter, Go
- GitHub Copilot, Gemini CLI, Antigravity
- アジャイル開発, スクラム

### 運営・主催

- GDGoC Hakodate

### 趣味

- マラソン, スキー
- 航空管制, 飛行機乗り…etc

# 1. 本日のゴール (Workshop Goals)

---

AIがコードを自動生成する時代において、エンジニアやプランナーが持つべき**マインドセット**と**本質的なスキル**を定義します。

- ①「コードが書けること」は重要ではない
- ②「作りたいもの」を詳細に言語化できるか
- ③「モデル」と「エージェント」の違い
- ④「無料」を使い倒せ & アカウントの特権を活かせ
- ⑤ AIへの課金は「自己投資」

## ①「コードが書けること」は重要ではない

---

- **背景:** 生成AI(LLM)の進化により、プログラミング言語の文法知識や単純なコーディング作業の価値は相対的に低下しています。
- **本質:** 私たちが目指すべきは「コードを書くこと」ではなく、\*\*「価値のあるプロダクト(動くソフトウェア)を世に送り出すこと」\*\*です。
- 開発プロセスの多くをAIに任せられる今、アイデアをカタチにする速度(タイム・トゥ・マーケット)が最大の競争力になります。

## ②「作りたいもの」を詳細に言語化できるか

---

- **課題:** AIに「〇〇なアプリを作って」と曖昧(抽象的)に頼むと、AIは一般的な無難なものしか出力できず、期待通りの成果物は得られません。
- **具体化の技術(インプットの重要性):**
  - ターゲットユーザー、解決したい課題、具体的な機能要件(Input/Output)
  - 画面構成、表示するデータ(APIの仕様など)
  - これらを**具体的かつ論理的に言語化**する力が必要です。
  - プロンプトにどれだけ\*\*「具体的で詳細な文脈(Context)」\*\*を与えられるかが、成果物のクオリティを左右します。

### ③「モデル」と「エージェント」の違い

---

- **モデル (Model)**: 知識や推論能力を提供する「脳」にあたる存在。
  - 例: GPT-5.4, Claude Sonnet 4.6, Gemini 3.5 Flash
- **エージェント (Agent)**: 脳の命令を受けて、実際にファイルを読み書きしたり、ターミナルでコマンドを実行したり、ブラウザを操作して検証を行う「手足」を持った自律プログラム。
- **現在地**: 単にチャットでAIと対話する時代から、**エージェントが開発環境と統合され、動くソフトウェアを自動で構築・検証してくれる時代へ。**

## ④「無料」を使い倒せ & アカウントの特権を活かせ

---

- **個人の無料枠:** Google AI Studio、Claude Consoleの初期枠など、無料プランを組み合わせることで強力な開発環境・API環境を構築可能です。
- **大学アカウントの活用（特権）:**
  - **MS Copilot 365（大学アカウント）:** 組織向けセキュリティ適用状態で、**GPT-5.4 / GPT-5.5** などの最新モデルを安全かつ無料で利用可能。コードが再学習に使われません。
  - **Google Workspace（大学アカウント）:** セキュリティが保証された環境で Gemini等の高性能モデルを安全に利用可能です。

## ⑤ AIへの課金は「自己投資」— 差別化と生産性

---

- **個人の自由と選択:** 課金は自由ですが、月額数千円(**飲み会1回分**)の投資で得られる生産性向上と学びは計り知れません。
- **将来へのアドバンテージと差別化:** 企業就職後も生成AIを使うのは当たり前。今のうちに有料ツールを使いこなすことで、「AIを実務に組み込むスキル」で他の人と圧倒的な差別化を図れます。
- **コピペ開発からの脱却:** Webチャットへのコピペでは、プロジェクト全体の依存関係や構成をAIが理解できません。
- **有料エージェント(Cursor, Claude Code, Antigravity)** を使うことで、**プロジェクト全体を丸ごと読み取って自動開発**する世界を体験できます。



## 2. モデル (Models)

各プラットフォームの最新概要（2026年6月11日現在）

## GPT (OpenAI) — 最新モデル: GPT-5 系列

---

- **GPT-5.5 / GPT-5.4**: 最上位フラッグシップモデル。最高精度の推論や複雑なエージェントタスクに対応。
- **GPT-5.4 mini**: 高速・低コスト。大量処理や日常的なコーディング補助向け。
- **利用ツール**:
  - **ChatGPT**: ブラウザ/デスクトップアプリから手軽に使える対話型AI。
  - **Codex / OpenAI Platform**: APIキーの発行、モデルのテストを行う開発者向け環境。

## Claude (Anthropic) — 最新モデル: 4～5系列

---

- **Claude Mythos**: 最上位・次世代最高推論。高度な知識労働や複雑な推論向け。
- **Claude Fable**: 高性能フラッグシップ。複雑なコーディング等に対応。
- **Claude Opus 4.7 / Sonnet 4.6 / Haiku**: 用途に応じた高性能・汎用モデル群。
- **最新機能・エージェント**:
  - **Claude Code**: ターミナル上で自律的に動作するコーディング専用CLIエージェント。
  - **Claude Cowork**: デスクトップアプリと連携し、ローカルやクラウドで自律実行。

## Gemini (Google) — 最新モデル: Gemini 3.5 系列

---

- **Gemini 3.5 Flash**: エージェント特化。コーディング・マルチエージェントワークフローでフロンティア性能を発揮。
- **Gemini 3.1 Pro / Deep Think**: 複雑な推論、科学・研究・工学向け最高推論。
- **利用ツール・環境**:
  - **Google AI Studio**: Gemini APIキーの無料発行とテストができる開発者向け platform。コーディング支援能力が高く無料枠が豊富。
  - **Google Antigravity**: Gemini 3.5 Flash搭載の自律型エージェント開発環境。

## 2.5. AIプラットフォームと利用チャネル

多様な利用形態(デバイス・統合環境)の理解

# 利用形態(チャンネル)ごとの代表例

| 利用形態            | 特徴              | 主要なツール・名称                                     |
|-----------------|-----------------|-----------------------------------------------|
| ブラウザ (Web UI)   | 対話型チャットUI       | ChatGPT, Claude.ai, Gemini, Copilot           |
| デスクトップ APP      | OS統合、画面共有など     | ChatGPT Desktop, Claude Desktop               |
| モバイル APP        | 移動中の使用、音声対話     | ChatGPT, Claude, Gemini, Copilot App          |
| IDE 拡張 / 専用 IDE | エディタ連携、リアルタイム開発 | GitHub Copilot, <b>Cursor</b> (AI-native IDE) |
| CLI / テーミナル     | ターミナル操作 自律実行    | <b>Claude Code, Gemini CLI,</b>               |

# 提供ツール・インターフェース対応表

| プラットフォーム    | ブラウザ<br>(Web)  | IDE拡張 / 専用<br>IDE | CLI / エージェント             | 開発者プラットフォーム       |
|-------------|----------------|-------------------|--------------------------|-------------------|
| OpenAI / MS | ChatGPT        | GitHub Copilot    | gh copilot               | OpenAI Playground |
| Anthropic   | Claude.ai      | Cursor            | Claude Code              | Anthropic Console |
| Google      | Gemini Web     | VS Code 拡張        | Gemini CLI / Antigravity | Google AI Studio  |
| MongoDB     | Atlas (Web UI) | Atlas Extension   | Atlas CLI                | MongoDB Atlas     |

# 3. エージェント & 開発環境

AIと協調する自律実行とエディタ



## エージェント & 開発環境の主要ツール

---

- **Claude Code**: ターミナル上で自律動作し、ファイル編集からテスト実行までこなすCLIEージェント。
- **Gemini CLI**: Gemini 3.5 Flashの力をコマンドラインから手軽に呼び出すツール。
- **Cursor**: 複数モデルを切り替え可能で、差分インライン編集やプロジェクト全体の参照が極めて強力なAIネイティブエディタ。
- **VS Code**: デファクト標準エディタ。Marp、Roo ClineなどのAI拡張を追加して強力な環境へ。

# 4. デザイン & 5. MCP

UIプロトタイプ生成と外部ツール連携規格

# デザインツール と MCP (Model Context Protocol)

---

- デザインプロトタイプ生成:
  - **Figma Make**: プロンプトからデザイン要素を自動生成する機能。
  - **Stitch**: UIコンポーネントの視覚的構築とコードエクスポート。
- **Model Context Protocol (MCP)**:
  - AIモデルが外部のデータソース、ツール、APIと通信する規格。
  - **Chrome DevTools MCP**: AIがブラウザを操作しコンソール等を確認。
  - **Figma MCP Catalog**: FigmaのデザインデータをAIが解釈して開発。
  - **MCP Atlas (MongoDB)**: マルチステップMCPの実行基盤。

# 6. 手を動かしてみよう

リアルタイム気象データ表示Webアプリの構築

## 演習課題の概要

---

- **目標:** AIEージェント(Cursor, Claude Code, Antigravityなど)を使い、動作するWebアプリケーションを短時間で構築する。
- **技術スタック:**
  - **フロントエンド:** React (Vite) + TypeScript + Tailwind CSS
  - **UIライブラリ:** shadcn/ui (Card, Button 等)
  - **グラフ描画:** Recharts (気温変化の視覚化)
  - **データソース:** Open-Meteo API (函館の7日間気温予測)

## プロンプト指示(例)

---

このデータを表示するWebアプリケーションを作りたい。

[フレームワーク1]: React (Vite)

[フレームワーク2]: TypeScript / Tailwind CSS

UIコンポーネントは shadcn/ui を使って。

データソースAPI:

`https://api.open-meteo.com/v1/forecast?latitude=41.841784&longitude=140.766926&hourly=temperature_2m&timezone=auto&past_days=0&forecast_days=7`

要件:

1. 上記API（函館の7日間の気温予測データ）からデータを非同期で取得し、見やすいグラフ（Rechartsなど）で視覚化すること。
2. shadcn/ui の Card や Button コンポーネントを使用し、ダッシュボード風のモダンなUIにすること。
3. 気温が特定のしきい値（例: 25度以上）を超える時間帯をハイライトする機能を付けること。

## ハンズオンの流れ

---

1. **プロジェクト初期化:** `npm create vite@latest` などを用いてエージェントにプロジェクトを作らせる。
2. **UIライブラリセットアップ:** `shadcn/ui` を導入し、テーマカラーにブランドカラー ( `#AF1F24` )を設定。
3. **API連携実装:** `fetch` / `axios` を使い、Open-Meteo APIから非同期でデータを取得。
4. **UI・ビジュアルライズ実装:** 7日間の気温予測をグラフ(Recharts)で表示し、しきい値(例: 25度以上)を超える時間帯をハイライト。
5. **ローカル確認:** `npm run dev` でローカルサーバーを起動し、動作確認。